

Assessment Task 2 - Regression Report

Predicting ORCL Next-Day Direction: A Classification Comparison of
K-Nearest Neighbors and Support Vector Machine

Student(s) Name(s):

Santiago Cruz Lopez | **ID:** 2584023

7CS070/UZ2: Concepts & Technologies of Artificial Intelligence

Online

Professor Vahid Rafe

August 24, 2026

Wolverhampton, UK

The University of Wolverhampton

Table of Contents

1. Introduction.....	3
2. Dataset and Method.....	3
3. Algorithm 1: K-Nearest Neighbors.....	4
4. Algorithm 2: Support Vector Machine.....	5
5. Results and Model Comparison.....	6
6. Performance Analysis.....	7
7. Suggested Improvements.....	9
8. Conclusion.....	10
References.....	11

1. Introduction

This report focuses on Task 2 of the portfolio, which involves building and comparing two classification algorithms using a real-world dataset. In Task 1, the goal was to predict the exact size of Oracle Corporation's (ORCL) next-day return. However, the analysis revealed almost no usable signal from eight technical indicator features. This task presents a simpler version of the same question: instead of predicting the magnitude of the return, can we at least predict the direction — whether it will go up or down?

To explore this, we will test two distinctly different algorithms, avoiding any assumptions about the outcomes. Each algorithm is implemented in its own independent notebook, allowing for individual verification of results before the two algorithms are compared here.

2. Dataset and Method

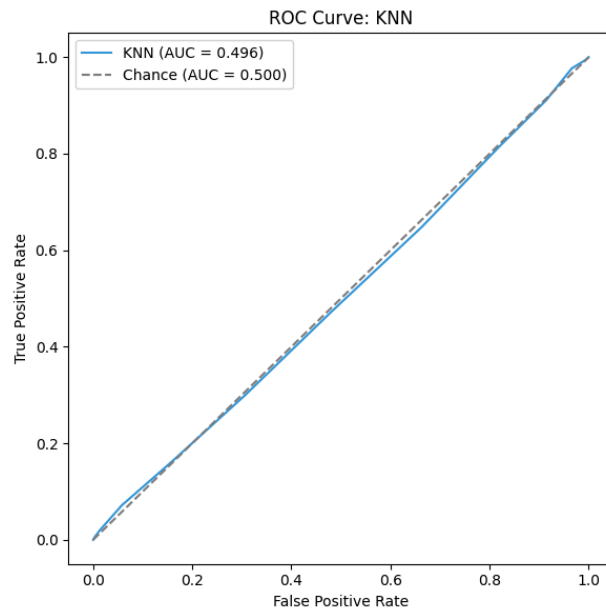
The dataset and the eight engineered features, which include daily return, 10-day volatility, percentage distance from three moving averages, and the RSI/TSI oscillators, remain the same as in Task 1. The target variable, called `target_direction`, is binary: it is set to 1 if ORCL closes higher the next day and 0 otherwise.

In both notebooks, every feature is scaled using `StandardScaler` before training. This scaling is applied only to the training partition because both K-Nearest Neighbors and the Support Vector Machine with an RBF kernel measure the distance between data points internally. Without scaling, a feature like `dist_sma252` (which has values around ± 0.3) could dominate a feature like `rsi` (which has values ranging from 0 to 100) solely due to its numeric scale, rather than its relevance.

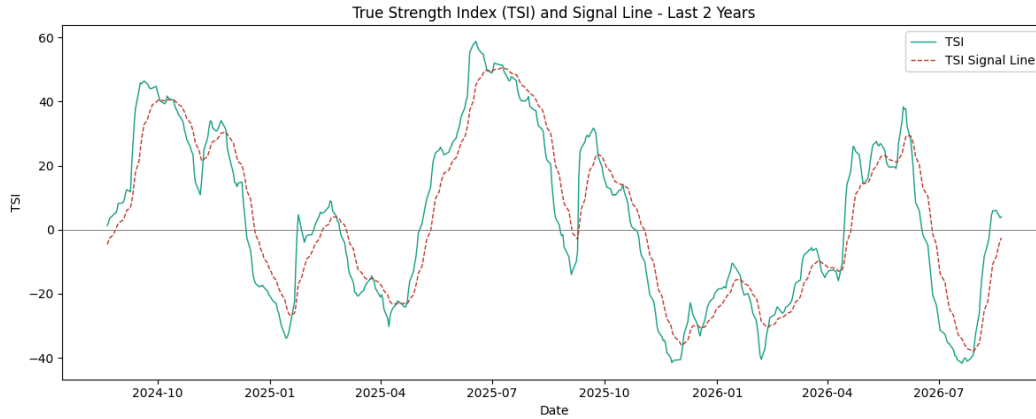
As in Task 1, the data is split chronologically, with 80% allocated for training and 20% for testing. This approach avoids random sampling to ensure that the model does not train on information from the future.

3. Algorithm 1: K-Nearest Neighbors

KNN classifies a new day by identifying its k most similar historical days, using Euclidean distance across eight scaled features. It then determines the predicted outcome by taking a majority vote of these historical outcomes (Cover & Hart, 1967). There is no explicit training phase; the "model" consists solely of the stored training data, with all computations occurring at the time of prediction. For this analysis, k was set to 15. A smaller value of k makes the model more sensitive to individual nearby points, leading to higher variance, while a larger k smooths predictions towards the overall class balance, resulting in higher bias (James et al., 2024).

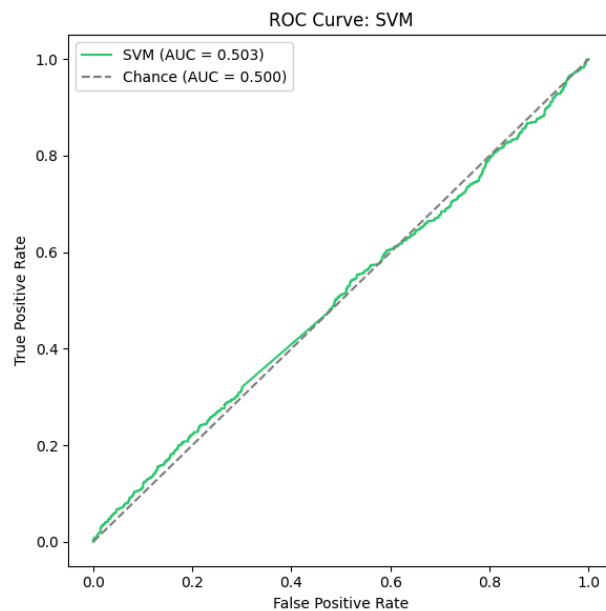


To illustrate the abstract concept of "distance across eight dimensions," the notebook includes a plot of the raw TSI indicator alongside its signal line for the past two years, highlighting their crossovers—a classic indicator of momentum shifts. The two lines closely follow each other throughout the period, providing visual confirmation that two of KNN's eight coordinates convey largely redundant information rather than independent signals.

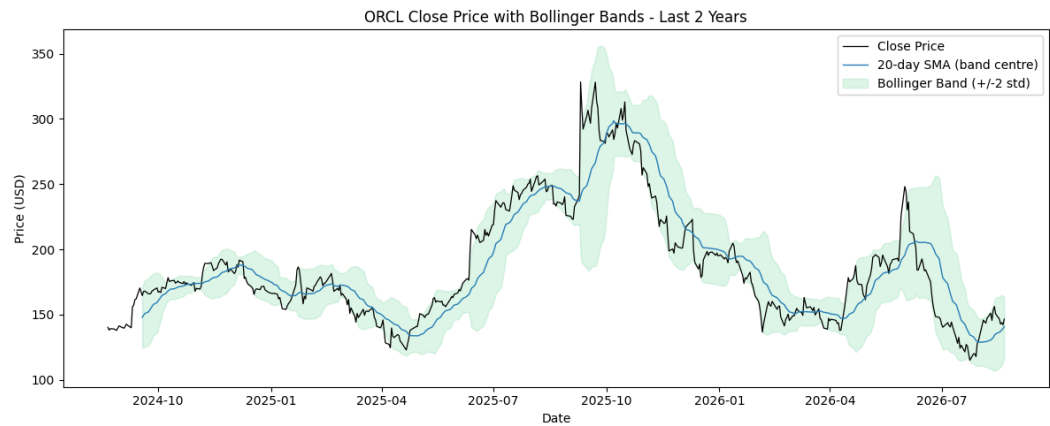


4. Algorithm 2: Support Vector Machine

A Support Vector Machine (SVM) seeks to find the boundary that separates two classes with the maximum possible margin (Cortes & Vapnik, 1995). The Radial Basis Function (RBF) kernel used in this context allows the boundary to curve by implicitly mapping the features into a higher-dimensional space. In this space, a straight separating boundary corresponds to a curved one in the original feature space, utilizing what is known as the "kernel trick." This technique avoids the need to explicitly compute the coordinates in the higher-dimensional space. Setting C to 1.0 balances achieving a wider margin while allowing for some misclassified training points. The parameter gamma set to 'scale' determines the extent of influence that a single training example has.



The same notebook calculates Bollinger Bands, which are volatility-based margins drawn around a 20-day moving average. These bands serve as a visual analogy to the SVM's objective of maximizing the margin, rather than being a feature trained by the model. The bands visibly widen during periods of high price volatility, demonstrating why a single fixed-margin SVM may perform better in some timeframes than in others.



5. Results and Model Comparison

Table 1 summarises both models on the held-out test set, alongside the majority-class baseline.

Model	Accuracy	Balanced Accuracy	F1 (macro)	ROC-AUC	MCC
Baseline (majority)	47.5%	50.0%	—	—	0.000
KNN	49.5%	49.6%	49.5%	0.496	−0.008
SVM	50.6%	50.6%	50.5%	0.503	0.011

The Support Vector Machine (SVM) outperforms the K-Nearest Neighbors (KNN) on every metric, although both methods are nearly at the 50% chance level. Notably, KNN has a negative Matthews correlation coefficient (MCC), which indicates that it performs slightly worse than random guessing according to that measure.

In each notebook's 5-fold cross-validation section, this trend is reinforced: KNN achieves an average cross-validation accuracy of 50.4% (with a variation of ± 1.3 points across folds), while SVM achieves 51.2% (with a variation of ± 1.1 points).

Algorithm 1: K-Nearest Neighbors

5-fold TimeSeriesSplit cross-validation for KNN:

- Mean CV Accuracy: 0.5043 (std 0.0134)
- Mean CV Balanced Accuracy: 0.5042 (std 0.0136)

Algorithm 2: Support Vector Machine

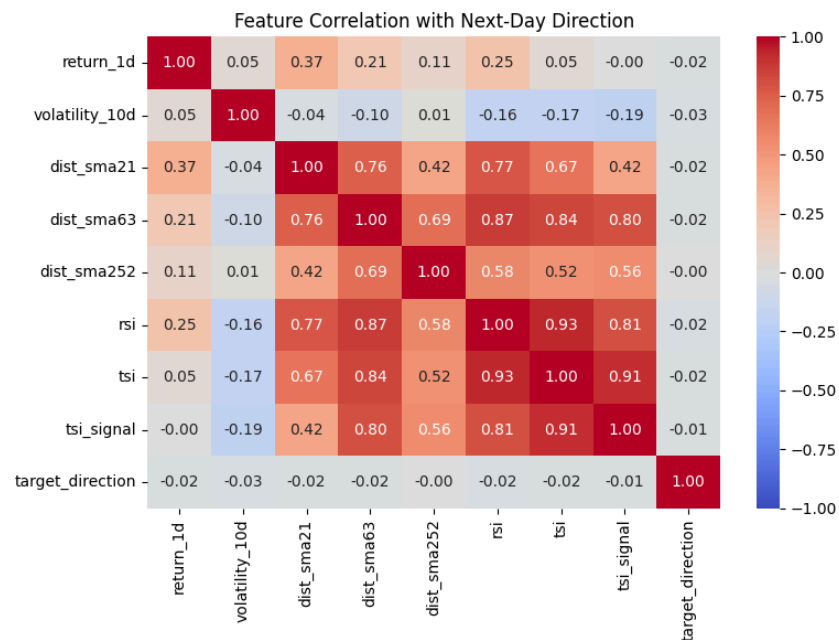
5-fold TimeSeriesSplit cross-validation for SVM:

- Mean CV Accuracy: 0.5123 (std 0.0109)
- Mean CV Balanced Accuracy: 0.5117 (std 0.0122)

The advantage of SVM is small but consistent across both evaluation methods. This is in contrast to the results from Task 1 with the Random Forest, where the rankings based on a single split and cross-validation disagreed on one metric.

6. Performance Analysis

The slight but consistent advantage of SVM can be attributed to the same underlying issue identified in Task 1: none of the eight features have a meaningful relationship with the target variable. The feature with the strongest single correlation, volatility_10d, has a correlation coefficient (r) of approximately -0.029, which is close to zero. Additionally, the boxplots in Section 3.1 demonstrate that the distributions of "Up" and "Down" for the three most correlated features overlap almost entirely.



Since there is no significant signal to capitalize on, the differences in the algorithms' mechanisms account for the small performance gap between them. KNN makes predictions based on a majority vote from the 15 most similar historical days. In Section 8's TSI deep-dive, it is shown that two of the eight distance coordinates (tsi and tsi_signal) have a high correlation of 0.91 with each other. Additionally, Section 3.2's complete correlation table indicates that this issue extends further, with correlations of 0.93 between rsi and tsi, and 0.87 between dist_sma63 and rsi.

Table 4. Section 3.2 (identical in both notebooks)

	return_ 1d	volatility_1 0d	dist_sma 21	dist_sma 63	dist_sma2 52	rsi	tsi	tsi_sigh al
return_1d	1	0.05	0.37	0.21	0.11	0.25	0.05	0
volatility_ 10d	0.05	1	-0.04	-0.1	0.01	-0.1 6	-0.1 7	-0.19
dist_sma2 1	0.37	-0.04	1	0.76	0.42	0.77	0.67	0.42
dist_sma6 3	0.21	-0.1	0.76	1	0.69	0.87	0.84	0.8
dist_sma2 52	0.11	0.01	0.42	0.69	1	0.58	0.52	0.56
rsi	0.25	-0.16	0.77	0.87	0.58	1	0.93	0.81
tsi	0.05	-0.17	0.67	0.84	0.52	0.93	1	0.91
tsi_signal	0	-0.19	0.42	0.8	0.56	0.81	0.91	1

KNN measures distance using fewer independent dimensions than its eight nominal features might suggest, which can reduce the quality of its "nearest neighbors." In contrast, SVM uses a margin-based boundary that is less affected by this redundancy. The RBF kernel operates on the overall geometry of the feature space rather than treating each dimension as an equally weighted

independent factor. This provides a plausible, evidence-based explanation for SVM's consistent, albeit small, advantage. It's important to note that this does not imply that SVM is inherently a superior algorithm in all cases.

7. Suggested Improvements

Three concrete changes can be made based on the evidence presented above.

First, we should remove redundant features. Specifically, the features "tsi" and "tsi_signal" are nearly duplicates of "rsi" (as discussed in Section 3.2). Eliminating two of the three oscillators would help mitigate KNN's redundant-dimension problem without requiring any additional retraining.

Second, we need to revisit the classification threshold and the target definition. Currently, the "target_direction" considers any positive return, no matter how small, as "Up." Implementing a threshold-based label — for example, only counting movements greater than $\pm 0.5\%$ as a genuine directional signal and categorizing smaller movements as noise — would eliminate borderline cases that are inherently uncertain, similar to the more deliberate $\pm 5\%$ threshold that was used for the regression labels in Task 1.

Third, we need to include data that is currently missing from the feature set. Notably, there isn't a trading-volume column available in this export. A volume-based confirmation signal is a standard technical analysis complement to the price-based indicators we are using. Additionally, the SVM's C and gamma hyperparameters, as well as KNN's k, were initially set by convention rather than being fine-tuned. Conducting a grid search with the same TimeSeriesSplit cross-validation already applied in Section 9 of each notebook would be a logical and low-risk next step (Hastie et al., 2009).

8. Conclusion

Both KNN and SVM were implemented independently in the notebooks **2584023_Task2_Algo1.ipynb** and **2584023_Task2_Algo2.ipynb**, using the same preprocessing methods, features, and evaluation techniques. Each model was cross-validated in isolation. SVM consistently outperformed KNN, albeit only slightly. This advantage can be attributed to SVM's ability to manage the feature redundancy identified in Task 1. However, neither model achieved performance significantly above chance level, which aligns with the regression findings from Task 1 and supports the notion of weak-form market efficiency (Fama, 1970). This does not indicate any flaws in either implementation. The evidence regarding correlation, multicollinearity, and threshold indicators gathered from both notebooks—including the detailed analyses of the TSI and Bollinger Bands—suggests concrete next steps rather than a dead end.

References

- Cortes, C., & Vapnik, V. (1995). *Support-vector networks* | *Machine Learning* | Springer Nature. Springer Nature. Retrieved August 24, 2026, from <https://link.springer.com/article/10.1007/BF00994018>
- Cover, T., & Hart, P. (1967). *Nearest neighbor pattern classification* | *IEEE Journals & Magazine*. IEEE Xplore. Retrieved August 24, 2026, from <https://ieeexplore.ieee.org/document/1053964>
- Hastie, T., Tibshirani, R., & Friedman, J. H. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition*. Springer.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2024). *An Introduction to Statistical Learning: With Applications in Python*. Springer International Publishing.